

Notes:

1. Look, I know it's a general thing, but the repetition of style is bound to get boring, it's common and I get it but I can very much see it becoming a problem. The tables are one of the things kinda getting repetitive the most.
 - a. What do I suggest? Let me take a comparison.

<code>mut_plus(double, Unit)</code>	Increments the internal value by an amount in another unit. The internal unit will stay the same
<code>mut_plus(Measure)</code>	Increments the internal value by another measurement. The internal unit will stay the same
<code>mut_minus(double, Unit)</code>	Decrements the internal value by an amount in another unit. The internal unit will stay the same
<code>mut_minus(Measure)</code>	Decrements the internal value by another measurement. The internal unit will stay the same
<code>mut_times(double)</code>	Multiplies the internal value by a scalar
<code>mut_divide(double)</code>	Divides the internal value by a scalar
<code>mut_replace(double, Unit)</code>	Overrides the internal state and sets it to equal the given value and unit
<code>mut_replace(Measure)</code>	Overrides the internal state to make it identical to the given measurement
<code>mut_setMagnitude(double)</code>	Overrides the internal value, keeping the internal unit. Be careful when using this!

Constant	Set to	What it is
<code>Drive.DEADBAND</code>	<code>0.10</code>	lesson 01's threshold
<code>Drive.MAX_VOLTS</code>	<code>12.0</code>	most voltage we ever command
<code>Drive.GEAR_RATIO</code>	<code>8.45</code>	motor turns per wheel turn
<code>Flywheels.MAX_RPM</code>	<code>5800.0</code>	motor free speed

- i. First of all, I believe we can remove the titles at the top, try to keep it clean and just state the constant and its purpose in a more simplistic, elegant way I suppose. For example, `mut-divide(double)`'s explanation has the word scalar in it, now scalar could confuse someone but I believe that changing it to something like "single value" could do the job. Try to

remove the “fluff” and change max_volts’s definition to Maximum voltage.

- ii. Secondly, almost every single lesson starts off with some sort of table or repetition of the last lesson format and it just gets boring sometimes.

- 2. My suggestion is to introduce code! Going back to my old comparison right below we have examples:

```
MutDistance measure = Feet.mutable(0);
measure.mut_plus(10, Inches); // 0.8333 feet
measure.mut_plus(Inches.of(10)); // 1.6667 feet
measure.mut_minus(5, Inches); // 1.25 feet
measure.mut_minus(Inches.of(5)); // 0.8333 feet
measure.mut_times(6); // 0.8333 * 6 = 5 feet
measure.mut_divide(5); // 5 / 5 = 1 foot
measure.mut_replace(6.2, Meters) // 6.2 meters - note the unit changed!
measure.mut_replace(Millimeters.of(14.2)) // 14.2mm - the unit changed again!
measure.mut_setMagnitude(72) // 72mm
```

i.

- b. Use comments, show how if you were to change the constant in some different way that’s not the way you’re asking and show examples. The biggest way you can show how code works is through comments and examples!

- 3. Ask them to show mistakes in code.

- a. We both know that bug fixing consists of 90% of coding time. So add some examples of code and use the idea that they have to show some sort of knowledge towards you that they remember or are learning what they are being shown. It’s very easy to cheese a project and eventually get it right or just skipping to hints. Ask them to spot a bug and introduce good coding habits!

Can you spot the bug in this code?

```
private Distance m_lastDistance;
public Distance calculateDelta(Distance currentDistance) {
    if (m_lastDistance == null) {
        m_lastDistance = currentDistance;
        return currentDistance;
    } else {
        Distance delta = currentDistance.minus(m_lastDistance);
        m_lastDistance = currentDistance;
        return delta;
    }
}
```

i.

- ii. Quiz them on the constants they are learning and introduce problems relating to the definition of the constants. Get them into the mindset that they have to imagine an input in the code and get an output, that mindset is the basis of how every coder should think and debug their code.

- 4. Highlight important parts of the code or pieces of code that are relating to the problem, use pictures, relate code to the current game, don’t space them out!

- a. Coding is not just learning how to program but integrating code into real world problems. You have to know the game, the robot, and the fundamentals of how

certain things work in order to program well. Introduce topics, try to get them to visualize the process of coding rather than coding itself.

- b. One of the biggest things that I struggled with when I first started programming is that I just started blindly programming and didn't really understand what I was aiming for or going for. Introduce them into the robot, tell them how the game works, what each mechanism is needed for and what its intended use is, introduce motor names, technical terms, get them familiar with the "robotics lingo" so they actually know. You must establish the connection between FRC and Programming because every subteam is connected in some way.

No cloud service that can be down on the day of your meeting.

5.

- i. Down? Is that supposed to be done? Wording sometimes just feels clunky or sluggish.

ii. **Twenty minutes becomes two.**

- iii. AI just becomes apparent at a certain point and this basic format of wording becomes super SUPER repetitive. Click through each of the lessons and almost every single one of them looks the same and it gets boring.

- 6. Include simulations! I believe that the project idea is a great idea and I love the idea of having a walkthrough project with it guiding you. But there is not results, just a check and along with this it just feels like a pointless personal project at a certain point

- 7. One of the biggest things is that I don't feel it's beginner friendly at all!

This is gonna be one of my biggest points. I'm mainly comparing this to the FRC WPI Docs because in the beginning it starts by explaining what WPI Lib is, explaining elements of WPI Lib, showing screenshots and highlighting important features that you should be seeing. In your website, you jump people right into the programming portion telling them to download things they probably have never heard before and giving them little to no information about anything! Along with this, you're giving them the same repetitive formula that doesn't explain anything. I assume that beginners means little to no programming experience at all and this definitely does not do a good job at explaining the intricacies in programming. You must explain everything, what is WPI Lib? Gradle? JUnit, the WPILib simulator, and AdvantageScope? You don't do it at all and when I looked at the supposed "handbook" it

just brought me back to the lessons that talked me through a project and vague descriptions.

(WILL ADD MORE ALSO PLEASE CALL ME TO BETTER EXPLAIN INSTEAD OF WRITING IT OUT)